COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/09/26	**DATE SUBMITTED:**	07/09/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 **Hands-on Activity 1.1 Basic C++ Programming**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a function prototype. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

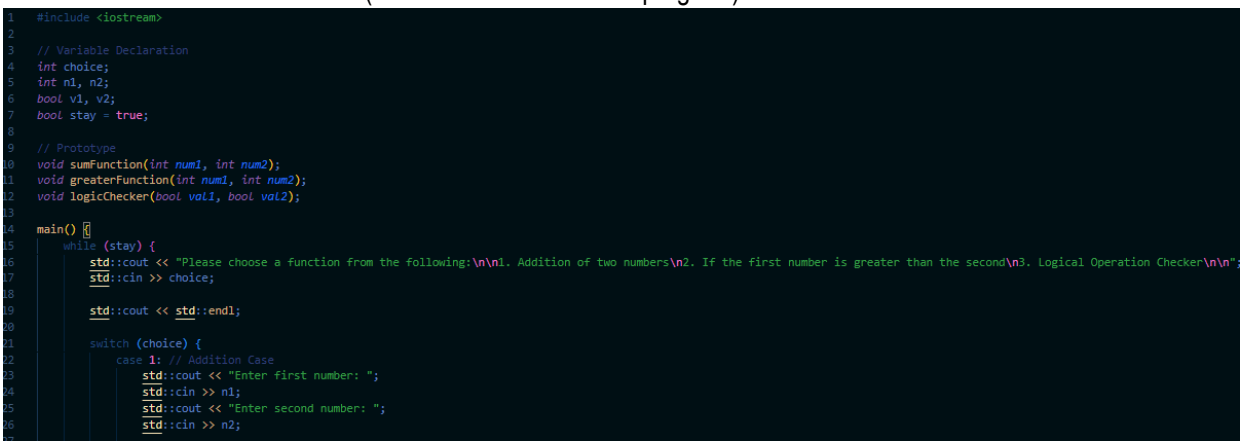
A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

Note:

- The driver program must call each function.
- The definitions must be after the main function.

❖ ILO A OUTPUT (All three functions in one program):



```
#include <iostream>

// Variable Declaration
int choice;
int n1, n2;
bool v1, v2;
bool stay = true;

// Prototype
void sumFunction(int num1, int num2);
void greaterFunction(int num1, int num2);
void logicChecker(bool val1, bool val2);

main() {
    while (stay) {
        std::cout << "Please choose a function from the following:\n\n1. Addition of two numbers\n2. If the first number is greater than the second\n3. Logical Operation Checker\n\n";
        std::cin >> choice;

        std::cout << std::endl;

        switch (choice) {
            case 1: // Addition Case
                std::cout << "Enter first number: ";
                std::cin >> n1;
                std::cout << "Enter second number: ";
                std::cin >> n2;
```

```
                sumFunction(n1, n2);
                break;

            case 2: // Comparison Case
                std::cout << "Enter first number: ";
                std::cin >> n1;
                std::cout << "Enter second number: ";
                std::cin >> n2;

                greaterFunction(n1, n2);
                break;
            case 3: // Logical Operation
                std::cout << "Enter first boolean value (1 for True / 0 for False): ";
                std::cin >> v1;
                std::cout << "Enter second boolean value (1 for True / 0 for False): ";
                std::cin >> v2;

                logicChecker(v1, v2);
        }

    std::cout << "\nContinue? (1 for Yes / 0 for No): ";
    std::cin >> stay;
    }

    std::cout << "\nHave a nice day!";
    return 0;
}

// Function Definitions

void sumFunction(int num1, int num2) {
    int total = num1 + num2;
    std::cout << "\nAnswer: " << total;
}

void greaterFunction(int num1, int num2) {
    if (num1 > num2) {
        std::cout << "\nThe first number is GREATER THAN the second.";
    } else {
        std::cout << "\nThe first number is LESS THAN the second.";
    }
}

void logicChecker(bool val1, bool val2) {
    std::cout << "\nAND operation: " << (val1 && val2) << "\n";
    std::cout << "OR operation: " << (val1 || val2) << "\n";
    std::cout << "NAND operation: " << (!(val1 && val2)) << "\n";
    std::cout << "OR operation: " << (!(val1 || val2)) << "\n";
    std::cout << "NOT (First Value): " << (!val1) << "\n";
    std::cout << "NOT (Second Value): " << (!val2) << "\n";
    std::cout << "XOR operation: " << (val1 != val2) << "\n";
}

/* CODE ANALYSIS:
    In the following code, I was tasked with creating 3 functions with their prototypes and definitions.
    I thought of the most efficient way to call all three and decided on using a while-loop to be able
    to choose between all three repeatedly. I also made use of the switch case which was very convenient
    with the menu system I made. Overall, the code runs as intended and fits exactly what I wanted to achieve.
*/
```

CODE ANALYSIS ABOVE

ILO A Test Output:

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 1.1> cd "c:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 1.1\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Please choose a function from the following:

1. Addition of two numbers
2. If the first number is greater than the second
3. Logical Operation Checker

1

Enter first number: 6
Enter second number: 7

Answer: 13
Continue? (1 for Yes / 0 for No): 1
Please choose a function from the following:

1. Addition of two numbers
2. If the first number is greater than the second
3. Logical Operation Checker

2

Enter first number: 6
Enter second number: 7

The first number is LESS THAN the second.
Continue? (1 for Yes / 0 for No): 1
Please choose a function from the following:

1. Addition of two numbers
2. If the first number is greater than the second
3. Logical Operation Checker

3

Enter first boolean value (1 for True / 0 for False): 1
Enter second boolean value (1 for True / 0 for False): 0

AND operation: 0
OR operation: 1
NAND operation: 1
OR operation: 0
NOT (First Value): 0
NOT (Second Value): 1
XOR operation: 1

Continue? (1 for Yes / 0 for No): 0

Have a nice day!
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 1.1>
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements
In this section, the initial implementation for a class triangle will be implemented

Sections:	Answer:
Header File Declaration Section	
Global Declaration Section	None
Class Declaration and Method Definition Section	
Main Function	

```cpp
#include <iostream>
```

```cpp
// class name should follow PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
        //constructor
        Triangle(double A, double B, double C);

        //methods
        void setAngles(double A, double B, double C);

        //validation
        const bool validateTriangle();
};
```

```cpp
int main(){
    // Variable Declaration
    int x, y, z;

    //driver code
    std::cout << "Triangle Verifier\n\nInput the first angle: ";
    std::cin >> x;
    std::cout << "Input the second angle: ";
    std::cin >> y;
    std::cout << "Input the third angle: ";
    std::cin >> z;
    Triangle set1(x,y,z);
    if(set1.validateTriangle()){
        std::cout<<"\nThe shape is a valid Triangle"<<std::endl;
    } else{
        std::cout<<"\nThe shape is NOT a valid triangle. \n";
    }

    return 0;
}
```

Method Definition	

```cpp
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB= B;
    angleC = C;
    totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB= B;
    angleC = C;
    totalAngle = A+B+C;
}

const bool Triangle::validateTriangle(){
    // makes sure angles are positive and sum up to exactly 180 degrees
    if (angleA > 0 && angleB > 0 && angleC > 0 && totalAngle == 180) {
        return true;
    }
    return false;
}
```

ILO B output observations and comments (Code Analysis):

In the code, if I were to point out my observations, I'd start with the sort of 'locality' of the variables wherein most of the program operates within local states and properties in their classes. Additionally, I actually added a way for the user to interact with the program, by allowing them to decide what the angles are themselves instead of it just being fixed within the program. E.g:

```
ocuments\MEDINA CPE010 - CPE21S1\HOA 1.1\" ; if ($?) { g++ main
 if ($?) { .\main2 }
Triangle Verifier

Input the first angle: 60
Input the second angle: 60
Input the third angle: 60

The shape is a valid Triangle
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 1.1> cd
ocuments\MEDINA CPE010 - CPE21S1\HOA 1.1\" ; if ($?) { g++ main
 if ($?) { .\main2 }
Triangle Verifier

Input the first angle: 100
Input the second angle: 100
Input the third angle: 100

The shape is NOT a valid triangle.
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 1.1>
```

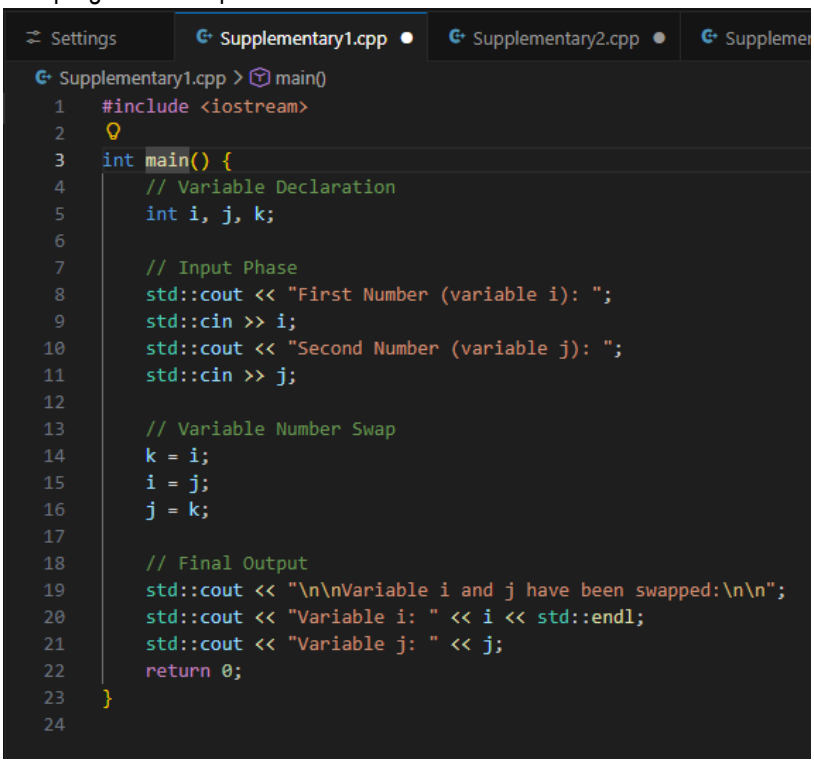
Lastly, as previously said, the program makes use of classes which kind of acts as a sort of blueprint that allows the functions to be built upon.

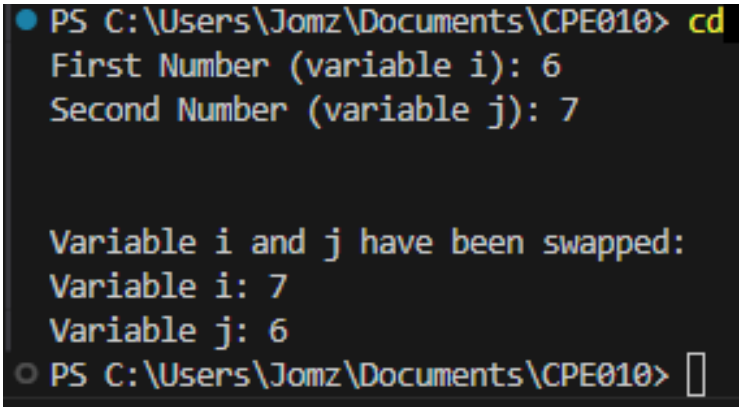
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve Different Problems using the C++ Programming Language
The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.



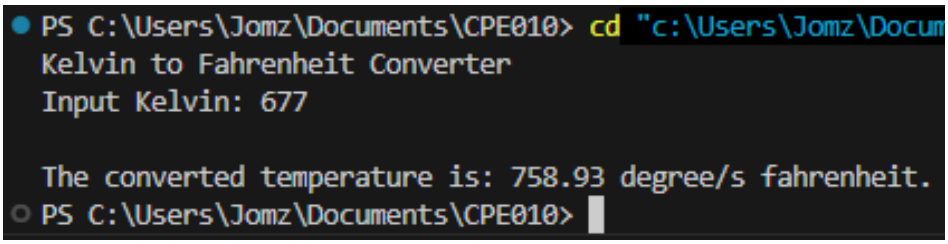


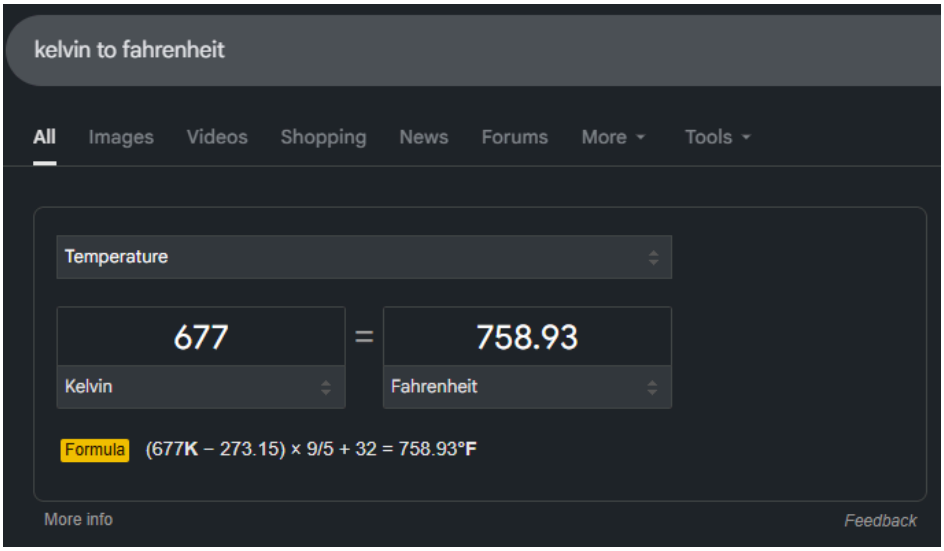
Code Analysis:

There's not much to explain with the code shown above. I simply facilitated the swapping of the numbers between two variables by making use of an extra variable and just assigning them around.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.



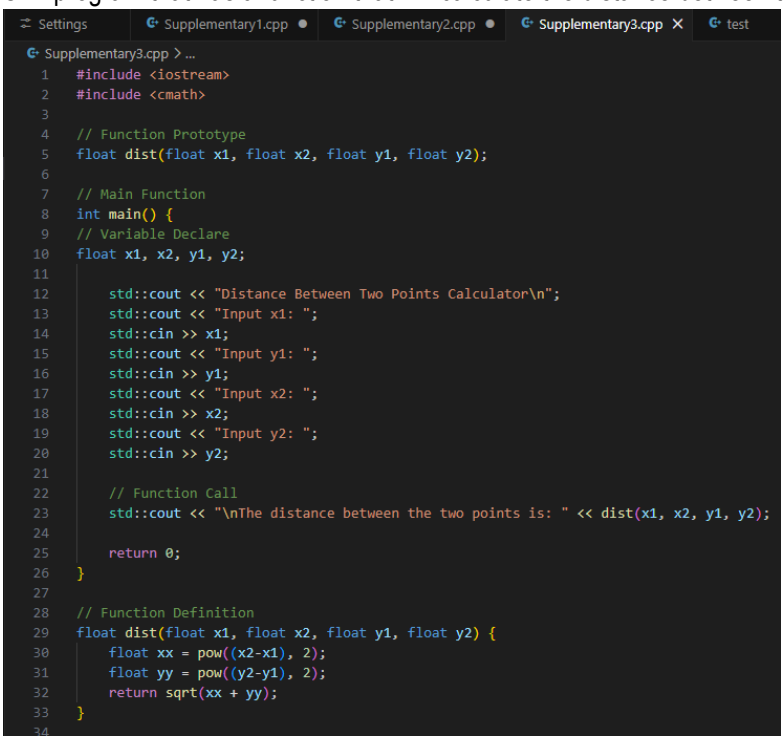


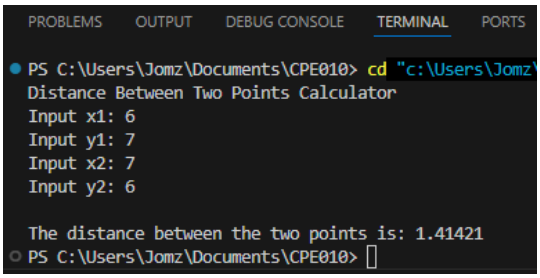


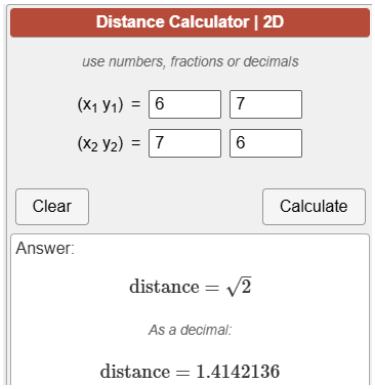
Code Analysis:

The code used is fairly straightforward. I simply stated the function prototype, used global float variables and the formula of converting kelvin to fahrenheit in the function definition, resulting in the answer by calling it in the main function output.

3. Create a C++ program that has a function that will calculate the distance between two points.







Code Analysis:

This program is basically the same with supplementary activity 2, just that I had to include the cmath library to handle powers and square roots as the formula needed it. I also had to separate the variables in the function definition because I kept getting an error.

4. Modify the code given in ILO B and add the following functions:
   A function to compute for the area of a triangle
   A function to compute for the perimeter of a triangle
   A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

Settings | Supplementary1.cpp | ILO_B.cpp | Supplementary2.cpp

ILO_B.cpp > setSides(double, double, double)

```cpp
#include <iostream>
#include <cmath>

// class name should follow PascalCase
class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
        float sideA, sideB, sideC; //for the peremeter

    public:
        //constructor
        Triangle(double A, double B, double C);

        //methods
        void setAngles(double A, double B, double C);

        //validation
        const bool validateTriangle();

        // Supple 4 prototypes
        void setSides(double a, double b, double c);
        float area();
        double perimeter();
        void triType();
};

int main(){
    // Variable Declaration
    int x, y, z;

    //driver code
    std::cout << "Triangle Verifier\n\nInput the first angle: ";
    std::cin >> x;
    std::cout << "Input the second angle: ";
    std::cin >> y;
    std::cout << "Input the third angle: ";
    std::cin >> z;
    Triangle set1(x,y,z);
```

```
int main(){
    if(set1.validateTriangle()){

        // the perimieter thing
        double a, b, c;
        std::cout << "\nEnter side A: ";
        std::cin >> a;
        std::cout << "Enter side B: ";
        std::cin >> b;
        std::cout << "Enter side C: ";
        std::cin >> c;


        set1.setSides(a,b,c);

        std::cout << "\nPerimeter: " << set1.perimeter() << std::endl;
        std::cout << "Area: " << set1.area() << std::endl;

        std::cout << "Triangle Type: ";
        set1.triType();
    } else{
        std::cout<<"\nThe shape is NOT a valid triangle. \n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB= B;
    angleC = C;
    totalAngle = A+B+C;
    sideA = sideB = sideC = 0; // reset
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB= B;
    angleC = C;
    totalAngle = A+B+C;
}
```

Settings | Supplementary1.cpp | ILO_B.cpp | Supplementary2.cpp | Supple

ILO_B.cpp > main()

```cpp
void Triangle::setAngles(double A, double B, double C){

}

const bool Triangle::validateTriangle(){
    // makes sure angles are positive and sum up to exactly 180 degrees
    if (angleA > 0 && angleB > 0 && angleC > 0 && totalAngle == 180) {
        return true;
    }
    return false;
}

// Supple 4 func definitions
void Triangle::setSides(double a, double b, double c){
    sideA = a;
    sideB = b;
    sideC = c;
}

double Triangle::perimeter(){
    return sideA + sideB + sideC;
}

float Triangle::area(){
    float s = perimeter() / 2;
    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

void Triangle::triType(){
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout << "Acute-angled" << std::endl;
    }
    else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout << "Obtuse-angled" << std::endl;
    }
    else{
        std::cout << "Others" << std::endl;
    }
}
```

```
PS C:\Users\Jomz\Documents\CPE010> cd "c:
Triangle Verifier

Input the first angle: 60
Input the second angle: 60
Input the third angle: 60

The shape is a valid Triangle


Enter side A: 100
Enter side B: 120
Enter side C: 100

Perimeter: 320
Area: 4800
Triangle Type: Acute-angled
PS C:\Users\Jomz\Documents\CPE010> cd "c:
Triangle Verifier

Input the first angle: 120
Input the second angle: 30
Input the third angle: 30

The shape is a valid Triangle


Enter side A: 50
Enter side B: 50
Enter side C: 50

Perimeter: 150
Area: 1082.53
Triangle Type: Obtuse-angled
PS C:\Users\Jomz\Documents\CPE010> cd "c:
Triangle Verifier

Input the first angle: 677
Input the second angle: 6777667
Input the third angle: 676677

The shape is NOT a valid triangle.
PS C:\Users\Jomz\Documents\CPE010>
```

Code Analysis:

The code above basically just follows ILO B but with the added function prototypes and definitions as told to do per the instructions. I used the inclusion of the header <cmath> in order to handle the square root for the area function and even more requests from the user to provide the perimeter, have a function that calculates for it, as well as deciding the type with if-else statements.

2. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Normal Google Search, Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Gemini was used in order to refresh and explain classes and how they were structured. It was also used with some slight bug fixing and proper code formatting.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.